

04.4 — Incident Response Plan

For: Structured response to outages — avoid 3 AM panic mode. **Goal:** Minimize MTTR (mean time to recover) by following a repeatable triage flow. **Last updated:** 2026-08-29 **Pre-req reading:** 00.2 Architecture Overview, 04.1 Deployment, 04.2 Daily Ops, 04.3 Disaster Recovery

Purpose

When something breaks at 3 AM, you don't want to be guessing. This runbook gives you:

- A **severity classification** so you know when to wake Venkat
- A **triage flow** for the first 15 minutes
- An **escalation matrix** with decision rules
- **Communication templates** for users + Venkat
- A **case study** (2026-08-29 500-outage) showing the playbook in action

This is a single-operator environment. There's no on-call rotation — but the patterns here still apply when subagents are responding.

Severity levels

SEV	Definition	Examples	Response time	Who acts
SEV-1	Total outage — every request fails; users can't work	HTTP 500 on every page; DB unreachable; nginx 502	wake Venkat IMMEDIATELY	subagent + Venkat
SEV-2	Degraded — some features broken, workaround exists	Roster loads but Attendance fails; one report errors	1 hour	subagent first, Venkat if not resolved
SEV-3	Minor — cosmetic / non-blocking	typo, missing button, slow query	1 day	daily queue
SEV-4	Cosmetic — back of queue	alignment, color, copy	when capacity allows	back of queue

Decision rule: if a user CANNOT DO THEIR JOB, it's SEV-1 or SEV-2. If they CAN work but with annoyance, it's SEV-3 or SEV-4.

Triage — first 15 minutes

When you suspect an outage (or Venkat reports one):

Step 1 — Confirm the outage (2 min)

```
# 1. Curl from outside (use a different network path to avoid local-network issues)
curl -sS -o /dev/null -w "%{http_code}\n" https://pberpPROD.duckdns.org/
# expect: 200, 302, 401, 403 – anything except 5xx
```

```
# 2. Curl login endpoint specifically
curl -sS -X POST https://pberpPROD.duckdns.org/api/method/login \
  -H 'Content-Type: application/json' \
  -d '{"usr":"test","pwd":"test"}' | jq .
# expect: {"message":"Invalid login..."} or 401 – NOT 500
```

```
# 3. Check if it's just THIS user or everyone
#   → if only 1 user, it's a user account issue (SEV-3)
#   → if all users, it's infra (SEV-1/SEV-2)
```

If curl returns 500 → **outage confirmed**, proceed to Step 2.

Step 2 — Check container status (2 min)

```
# SSH into main VPS
ssh -i /root/.openclaw/ssh_key vijay@144.217.163.228
```

```
# Container status (look for "Exited", "Restarting", or missing containers)
docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}' \
  | grep -E 'erp-(prod|qa|dev)-(backend|db|scheduler|queue)-1'
```

```
# Recent restarts (Lesson #154: a few restarts = normal after deploy; many = crash loop)
docker inspect --format='{{.Name}}: {{.RestartCount}} restarts, started {{.State.StartedAt}}'
erp-prod-backend-1 erp-prod-db-1 erp-prod-scheduler-1
```

```
# Resource usage
docker stats --no-stream | grep erp-prod-
```

What to look for:

- Container in Restarting loop → check why (logs, exit code)
- RestartCount > 5 in last hour → crash loop
- High CPU/MEM → could be slow query, runaway process
- Container missing → was it accidentally removed?

Step 3 — Check recent changes (3 min)

```
# What deployed in the last 24h?
ssh vijay@144.217.163.228 'cd /home/vijay/frappeclaw/frappeclaw-data/workspace/projects/ha
git log --oneline --since="24 hours ago"
```

```
# What backup scripts ran recently?
ssh vijay@144.217.163.228 'ls -lt /home/vijay/backups/prod/ | head -3'
```

```
# Any cron jobs triggered?
ssh vijay@144.217.163.228 'grep -i backup /var/log/syslog | tail -10'
```

```
# Any infra changes (docker, compose, nginx)?
ssh vijay@144.217.163.228 'cd /home/vijay/frappeclaw && \
git log --oneline --since="24 hours ago" frappeclaw-compose.yml nginx-erp.conf 2>/dev/nu
```

If a deploy happened in last 24h → **suspect the deploy** first (Lesson #153: install-app + no restart is the classic case study).

Step 4 — Check error logs (3 min)

1. Docker logs (last 100 lines per container)

```
for c in erp-prod-backend-1 erp-prod-db-1 erp-prod-scheduler-1; do
  echo "=== $c ==="
  docker logs --tail 100 $c 2>&1 | grep -iE 'error|traceback|exit|fatal|critical' | tail -
done
```

2. Frappe tabError Log (last 1h)

```
DB_PASS=$(docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD)
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -D _b80f05e76a0dcaad -e "
SELECT method, error, COUNT(*) as cnt, MAX(timestamp) as latest
FROM `tabError Log`
WHERE timestamp > DATE_SUB(NOW(), INTERVAL 1 HOUR)
GROUP BY method, error
ORDER BY cnt DESC
LIMIT 10;"
```

3. Nginx logs (for SSL/404/502 issues)

```
docker logs --tail 50 nginx-proxy 2>&1 | grep -iE 'pberpPROD|error'
```

Pattern matching:

- ModuleNotFoundError → Lesson #153 (gunicorn preload, needs restart)
- Connection refused to MariaDB → DB container down
- OperationalError: (2002, ...) → DB host/network issue
- OperationalError: (1045, ...) → DB auth issue (Lesson #87 — scheduler)
- 502 Bad Gateway → nginx can't reach backend
- 504 Gateway Timeout → backend slow / hung
- SSL handshake failed → cert issue

Step 5 — Form hypothesis (3 min)

Based on Steps 1-4, form a hypothesis. Common patterns:

Symptom	Likely cause	Action
HTTP 500, ModuleNotFoundError in logs	Lesson #153 (gunicorn preload)	docker restart erp-prod-backend-1
HTTP 502, backend logs empty	nginx misconfig	check nginx config + reload
HTTP 503, backend container exited	crash	check logs, restart
Login returns 401 with right password	DB password drift (Lesson #154)	docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD
Everything slow, CPU high	runaway query / process	check docker stats + tabError Log for hung queries
Users get "site not found"	DNS issue	check DuckDNS update, ping domain

Step 6 — Decide action (2 min)

Based on hypothesis + severity:

- **SEV-1** with known fix → execute the fix immediately, notify Venkat during/after
- **SEV-1** with unknown cause → notify Venkat FIRST, then investigate together
- **SEV-2** with known fix → execute the fix, document, notify Venkat after
- **SEV-3/4** → add to queue, no immediate action

Hard rule: if the fix involves DROP TABLE, rm -rf, down -v, or anything irreversible → **STOP and notify Venkat BEFORE acting** (Lesson #72: parent-verify before claiming SUCCESS).

Escalation matrix

Dev env issue

Try yourself → if blocked, ask subagent → if still blocked, ask Venkat
Dev is the scratch space. Break it freely. No approvals needed.

QA env issue

Try yourself (carefully) → notify Venkat "QA issue, investigating" → fix → ack
QA mirrors prod. Be more careful but don't ask permission to investigate.

Prod issue

Always notify Venkat IMMEDIATELY (Telegram @Miles_Morales_12)
→ Even if fix is obvious, the notification is required (Rule #13 from MEMORY)
→ Format: "PROD {env} SEV-{N}: {symptom}. Hypothesis: {X}. Action: {Y}. OK to proceed?"
If Venkat unreachable for > 15 min during SEV-1 → execute safest known fix, document, notify when Venkat available.

Data loss risk

STOP ALL CHANGES IMMEDIATELY
→ Notify Venkat with "DATA LOSS RISK" prefix
→ Do not execute any destructive action
→ Wait for Venkat's explicit YES

This is **NEVER** auto-resolved. Lesson #72: parent-verify saved us during the 2026-08-29 outage (volume verify before restart).

Communication templates

Internal status (Telegram / Slack)

SEV-1 first ping:

□ PROD SEV-1: pberpPROD returning HTTP 500 on all requests.

□ Status:

- Confirmed via curl from outside (502, not 302)
- Started ~03:06 IST
- Containers: all up, no crash
- Last deploy: haritha_hospital install 2026-08-28 evening
- tabError Log: spike in ModuleNotFoundError

□ Hypothesis: Lesson #153 – gunicorn --preload sys.path freeze after install-app

□ Action plan:

1. Verify volume persistence (data safe)
2. docker restart erp-prod-backend-1 (gunicorn reimports with fresh sys.path)
3. Smoke test

🕒 ETA: 5-10 min

□ OK to proceed?

During (status update every 5 min if > 10 min incident):

□ pberpPROD SEV-1: 5 min in. Volume verify PASSED (data safe). Executing restart now.

Resolution / all-clear:

□ pberpPROD SEV-1 RESOLVED.

□ Summary:

- Downtime: ~10 min (03:06 → 03:17 IST)
- Users affected: 100% of web requests
- Data loss: NONE (volumes preserved)
- Root cause: gunicorn --preload sys.path freeze after haritha_hospital install

□ Fix applied: docker restart erp-prod-backend-1

□ Follow-ups:

- Post-mortem started (LEARNINGS #153 already documented)
- Runbook updated (04.1-deployment.md, 04.4-incident-response.md)

User-facing (if public-facing)

Initial (if users start asking):

Subject: [Haritha Hospitals] Brief system interruption - investigating

Team,

We're aware of an issue affecting the ERPNext system. Our team is investigating and will provide an update within 30 minutes.

If you have urgent work, please save locally and try again later.

Apologies for the inconvenience.

Resolution:

Subject: [Haritha Hospitals] System restored

Team,

The issue has been resolved. The system is fully operational.

Root cause: gunicorn service needed a restart after a routine app update.
Data: all preserved, no loss.

If you encounter any lingering issues, please report via the usual channel.

Case study — 2026-08-29 500-outage

The canonical case study. Read this BEFORE you respond to any SEV-1.

Summary

Both pberpprod and pberpdev returned HTTP 500 on every request from 03:06 IST to 03:17 IST. Root cause: gunicorn's `--preload` flag froze `sys.path` at container start; today's `haritha_hospital` install wrote a `.pth` file but gunicorn's process tree couldn't see it. Resolution: `docker restart erp-{dev,prod}-backend-1` — fresh gunicorn = fresh `sys.path` = `.pth` visible. Total downtime: ~10 min. Zero data loss.

Detection

- **03:06 IST** — Venkat reported via Telegram: “both pberpdev and pberpprod are down - internal server error”
- Subagent (main session) was already active from late-session work

Initial response (3 min)

Confirmed outage

```
curl https://pberpPROD.duckdns.org/ # 500
curl https://pberpDEV.duckdns.org/  # 500
```

Containers up (not infra)

```
docker ps --format 'table ...' | grep erp-prod # all "Up"
docker ps --format 'table ...' | grep erp-dev  # all "Up"
```

Recent deploy

```
cd /home/vijay/frappeclaw-data/workspace/projects/haritha-hospitals
git log --oneline --since="24h" # haritha_hospital install commits
```

Recent container restart?

```
docker inspect --format='{{.Name}}: started {{.State.StartedAt}}' erp-prod-backend-1
# → started 2 days ago (prod), 9 days ago (dev) — well before today's install
```

Diagnosis

Spawned `haritha-diag-500` subagent (premium Minimax). Subagent found:

Inside container — direct python works

```
docker exec erp-prod-backend-1 python -c "import haritha_hospital"
# → SUCCESS
```

But via web request — fails

```
curl https://pberpPROD.duckdns.org/ # 500
docker logs --tail 50 erp-prod-backend-1 | grep ModuleNotFoundError
# → ModuleNotFoundError: No module named 'haritha_hospital'
```

Conclusion: gunicorn's `sys.path` was stale. `.pth` file written by `bench install-app` was not visible.

Hypothesis confirmation

- **Gunicorn --preload:** imports app once at startup, then forks workers. `sys.path` is read-only after that point.
- **Today's install:** `haritha_hospital` was installed in BOTH prod and dev (via `bench install-app`). The `.pth` file is created, but gunicorn's frozen `sys.path` doesn't include it.
- **Result:** every worker process → `import haritha_hospital` → `ModuleNotFoundError` → HTTP 500.

This matches Lesson #46 ("After install-app, restart backend") but the symptom had never been triggered before (install-app on a running container with `--preload` gunicorn).

Pre-restart verification (Lesson #72)

Before restarting, verified data persistence:

```
# Spawned haritha-verify-volumes subagent
# Confirmed:
# - erp-{dev,prod}_sites (sites, app code, site_config) → named volume
# - erp-{dev,prod}_db-data (MariaDB data) → named volume
# - Writable layer (Docker diff) → empty (no data in ephemeral layer)
```

Venkat was concerned restart would wipe data. Volume verify proved otherwise. **Critical Lesson #72 in action** — parent-verify saved potential data loss.

Fix

Venkat approved. Subagent `haritha-restart-both` executed:

```
# 1. dev first (lower risk, validates fix)
docker restart erp-dev-backend-1
sleep 45
curl https://pberpDEV.duckdns.org/ # → 200 □
curl -X POST .../api/method/login -d '{"usr":"Administrator","pwd":"admin123"}' # → 200 □

# 2. prod (higher risk, only after dev proven)
docker restart erp-prod-backend-1
sleep 45
curl https://pberpPROD.duckdns.org/ # → 200 □
curl -X POST .../api/method/login -d '{"usr":"Administrator","pwd":"PUNJD5HMP5B0uWBkRgeo"}'
```

Prod login returned **401** despite correct password. This triggered Lesson #154 — MEMORY's DB password had drifted. (Resolved later by retrieving from container `.env`.)

Recovery

Env	HTTP /	Login	Note
pberpdev	200 ☐	200 ☐	Fully restored
pberpprod	200 ☐	401 ☐	App responsive (200 = haritha loaded), admin password drifted

Lessons learned

- **#153 (new):** gunicorn --preload + new Python package = backend container restart REQUIRED
- **#154 (new):** docker restart $\neq$ docker-compose down — restart preserves volumes, never use down -v
- **#72 (reinforced):** Parent-verify before claiming SUCCESS — volume verify saved potential data loss
- **#157 (new):** master data migration pattern (separate session, related)

Action items

- ☒ Add LEARNINGS.md #153
- ☒ Add LEARNINGS.md #154
- ☒ Document gunicorn restart in deployment runbook (04.1)
- ☒ Add case study to incident response runbook (this doc)
- ☐ Add automated restart to deploy script (post-incident)
- ☐ Periodic restart check (cron?) — consider but not urgent
- ☐ Update MEMORY.md with actual DB passwords (Lesson #154)

Full timeline (UTC + IST)

UTC	IST	Event
2026-08-28 ~16:00	2026-08-28 21:30	haritha_hospital install on pberpprod
2026-08-28 ~17:30	2026-08-28 23:00	haritha_hospital install on pberpdev
2026-08-28 21:36	2026-08-29 03:06	Venkat reports both envs down
2026-08-28 21:38	2026-08-29 03:08	Diag subagent spawned
2026-08-28 21:40	2026-08-29 03:10	Root cause identified
2026-08-28 21:41	2026-08-29 03:11	Volume verify subagent spawned
2026-08-28 21:45	2026-08-29 03:15	Venkat approved restart
2026-08-28 21:46	2026-08-29 03:16	docker restart erp-dev-backend-1
2026-08-28 21:47	2026-08-29 03:17	docker restart erp-prod-backend-1
2026-08-28 21:47	2026-08-29 03:17	Both envs HTTP 200
2026-08-28 21:50	2026-08-29 03:20	Prod login 401 noted (password drift)

Subagent RunIds

- haritha-diag-500 — diagnosis (premium Minimax)
- haritha-verify-volumes — volume persistence check
- haritha-restart-both — restart execution + verify

References

- Commit hashes (today): 4023ccd (skeleton), 93e8c48 (fixtures + hooks.py)
 - Memory log: memory/2026-08-29.md §“CRITICAL INCIDENT”
 - Full post-mortem: 05.2 Post-Mortem §“Example: 2026-08-29 500-outage”
 - Lessons: LEARNINGS.md #46 (existing), #72 (existing), #153 (new), #154 (new)
-

Common incident types + first response

HTTP 500 blanket (every request fails)

First response: check unicorn, restart backend (Lesson #153).

```
docker logs --tail 50 erp-prod-backend-1 | grep ModuleNotFoundError
# If found → docker restart erp-prod-backend-1
```

```
# If not ModuleNotFoundError, check other patterns
docker logs --tail 100 erp-prod-backend-1 | tail -30
```

HTTP 502 / 503 (bad gateway / service unavailable)

First response: check nginx + backend reachability.

```
# Can backend be reached from nginx?
docker exec nginx-proxy curl -sS -o /dev/null -w "%{http_code}\n" http://erp-prod-backend-1
# If 000/unreachable → backend down, restart it
# If 200 → nginx config issue
```

```
docker exec nginx-proxy nginx -t # validate nginx config
docker exec nginx-proxy nginx -s reload # reload if config changed
```

DB connection errors

First response: check MariaDB container + credentials.

```
docker ps | grep erp-prod-db-1
docker logs --tail 50 erp-prod-db-1
```

```
DB_PASS=$(docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD)
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -e "SELECT 1;"
# If "SELECT 1" works → DB fine, issue is elsewhere
# If fails → DB down or corrupted, restore from backup (see 04.3)
```

Slow queries / timeout

First response: check recent migration + running queries.

Long-running queries

```
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -e "
SELECT id, user, host, db, command, time, state, LEFT(info, 100) as info
FROM information_schema.processlist
WHERE command != 'Sleep' AND time > 5
ORDER BY time DESC;"
```

Recent schema changes?

```
docker exec erp-prod-backend-1 bash -c '
cd /home/frappe/frappe-bench &&
bench --site pberpprod.duckdns.org console
>>> frappe.reload_doc("Core", "DocType", "Activity Log")'
# (Lesson #45: console API changed in v16)
```

tabError Log for "timeout" patterns

```
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -D _b80f05e76a0dcaad -e "
SELECT method, error FROM \tabError Log\ WHERE error LIKE '%timeout%' ORDER BY timesta
```

401 / 403 (auth issues)

First response: check user account + password rotation.

Is it just one user or all users?

If one user: check User DocType, password reset

If all users: DB password drift (Lesson #154)

```
DB_PASS=$(docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD)
```

Compare to MEMORY.md – if drift, update MEMORY

SSL errors

First response: check cert validity + renewal.

```
echo | openssl s_client -servername pberpPROD.duckdns.org \
  -connect pberpPROD.duckdns.org:443 2>/dev/null | openssl x509 -noout -enddate
```

If expired or expiring soon:

```
ssh vijay@144.217.163.228 'docker exec nginx-proxy /app/force-renew'
```

Post-incident checklist

Within 24 hours of resolution:

- [] 1. Restore full functionality verified (curl + browser smoke)
- [] 2. Users notified (all-clear message)
- [] 3. Venkat acknowledged (final Telegram message)
- [] 4. Post-mortem started (template: 05.2-post-mortem.md)
- [] 5. LEARNINGS.md updated (if root cause is novel)
- [] 6. Runbook updated (if response revealed a gap)
- [] 7. MEMORY.md updated (if state changed, e.g., DB passwords)
- [] 8. Heartbeat cron resumed (if paused during incident)

The post-mortem is **mandatory** for SEV-1 + SEV-2. See 05.2 Post-Mortem for template + example.

References

- **Architecture:** 00.2 Architecture Overview
- **Deployment:** 04.1 Deployment Runbook
- **Daily ops:** 04.2 Daily Ops Runbook
- **Disaster recovery:** 04.3 Disaster Recovery Runbook
- **Post-mortem:** 05.2 Post-Mortem Template + Example
- **Lessons cited:** #46 (install-app restart), #72 (parent-verify), #79 (hardened backup), #87 (scheduler MySQL auth), #95 (no bench console -c), #114 (silent cron), #151-#157 (custom app)

Calm > clever. Triage > heroics. Document > forget.